Robotic Arm and Chassis Coordinated Control

Robotic Arm and Chassis Coordinated Control

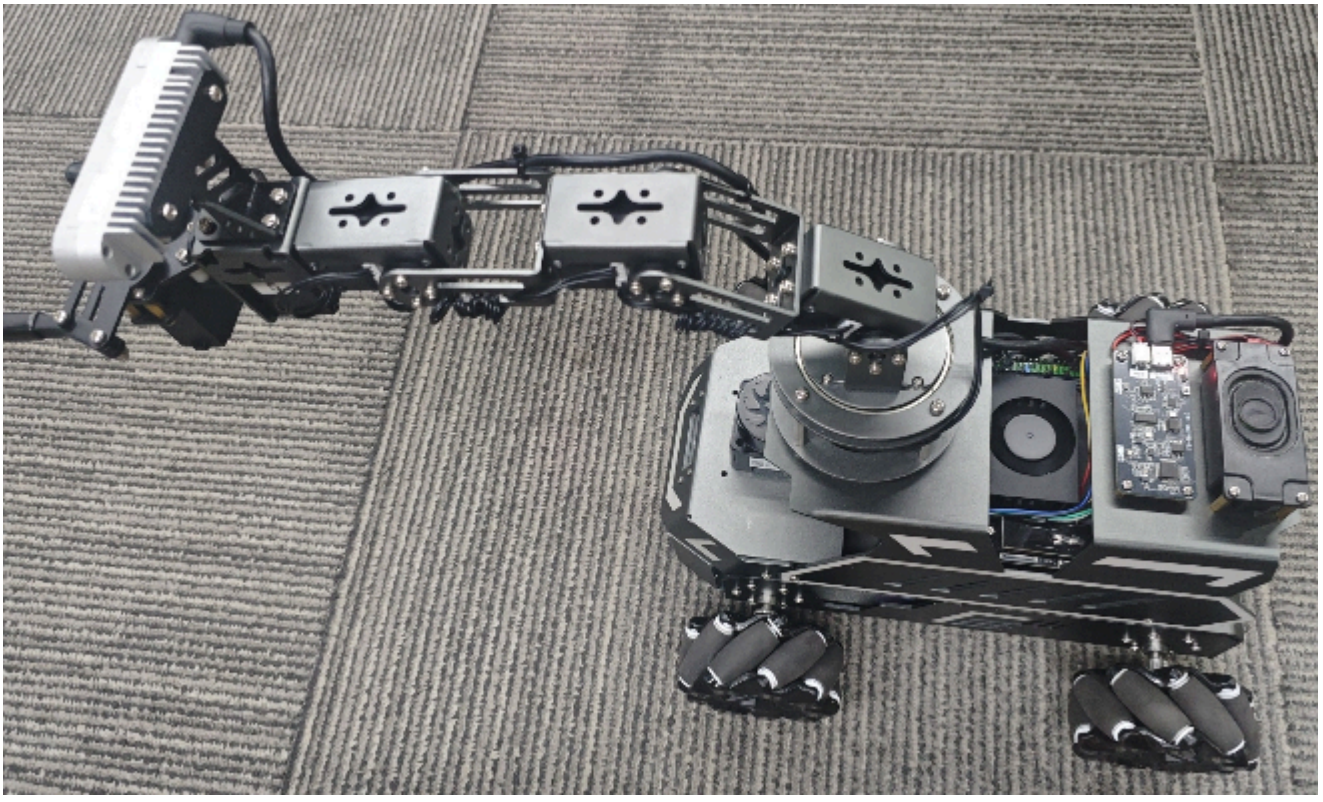
1. Content Description
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

1. Content Description

This course implements synchronized motion using the robotic arm's pose and the chassis, with the robotic arm and chassis making relative motions one after the other.

2. Preparation

```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=combined_control
```



3. Case Demonstration

- Trigger the action

```
ros2 service call /combined_move_trigger std_srvs/srv/Trigger {}
```

- Each time the action is triggered, both the robotic arm and the chassis move alternately in opposite

directions to achieve coordinated control



4. Core Source Code Analysis

Code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/arm_demo/combined_control.py
```

_trigger_callback function (service trigger callback):

- `self._running`: running status flag; if already running, reject new trigger requests
- `self._current_joint_states`: current robotic arm joint state; if not received, return an error
- `move_by_odom_client.wait_for_server(timeout_sec=3.0)`: waits for the chassis odometry Action server to be ready; if it times out at 3 seconds, returns failure
- `self._trigger_count`: trigger counter, used to alternately switch the motion direction
- `arm_direction = -1.0 if (trigger_count % 2 == 1) else 1.0`: on odd-numbered triggers the robotic arm moves backward (-x), and on even-numbered triggers the robotic arm moves forward (+x)
- `base_direction`: chassis motion direction, opposite to the robotic arm direction (when the robotic arm moves backward the chassis moves forward, and vice versa)
- `threading.Thread(target=self._combined_worker, daemon=True)`: starts a background daemon thread to execute the coordinated motion, avoiding blocking the service callback

_combined_worker function (worker thread wrapper):

- Calls `_combined_move_impl` to execute the actual coordinated motion, catches exceptions, and logs them

- `finally: self._running = False`: regardless of success or failure, resets the running status flag at the end

_combined_move_impl function (coordinated motion implementation):

- `base_j2, base_j3, base_j4`: extracts the J2/J3/J4 joint angles from the initial pose as reference values
- `fixed_j1, fixed_j5, fixed_j6`: the fixed and unchanging J1/J5/J6 joint angles
- `start_disp = self._arm_disp`: the starting extension amount is the current value, achieving continuous motion across triggers (not returning to the initial position)
- `end_disp`: target extension amount; when moving backward it is `-arm_distance`, and when moving forward it is `0.0` (the frontmost)
- `MoveByOdom.Goal()`: constructs the chassis odometry motion goal, where `distance` is the movement distance (meters) and `direction` is the direction (forward/backward)
- `send_goal_async(goal, feedback_callback=...)`: asynchronously sends the chassis motion goal and registers a feedback callback to update the moved distance in real time
- `goal_handle.accepted`: checks whether the goal is accepted by the server
- `progress = self._moved_distance / self.base_distance`: calculates the chassis motion progress (0~1), used to interpolate the robotic arm extension target
- `self._update_arm_reach(start_disp + (end_disp - start_disp) * progress, ...)`: linearly interpolates the robotic arm extension amount according to the chassis progress, achieving synchronized coordinated control of the chassis and the robotic arm
- Convergence phase after the chassis is in position: continuously issues the final extension command until the smoother output converges (difference less than 0.005), waiting at most 5 seconds
- `self._arm_disp = end_disp`: updates the current extension amount to prepare for the next trigger

_odom_feedback_callback function (odometry feedback callback):

- `self._moved_distance = feedback_msg.feedback.moved_distance`: updates the chassis moved distance in real time, used to calculate the coordinated progress

_update_arm_reach function (robotic arm extension update):

- `reach_pid.compute(displacement, 0.0)`: the PID controller calculates the raw output value according to the target extension amount
- Smoothing (Slew Rate Limiter): limits the change per step to no more than `max_step_change_z` to prevent robotic arm jitter
- `target_j2/j3/j4 = base_j2/j3/j4 + angular_z * gain_reach_j2/j3/j4`: distributes the PID output proportionally to the three joints J2/J3/J4 to achieve end-effector extension motion
- `max(0, min(180, target_j))`: limits the joint angle to the range of 0~180 degrees
- `pubSix_Arm(joints, runtime=runtime)`: publishes the 6-joint control command to the robotic arm driver
- `self._current_joints_cmd`: records the currently issued joint angle command

```
    def _trigger_callback(self, request, response):
        if self._running:
```

```
            response.success = False
            response.message = "Combined motion already running, please wait"
            return response
        if self._current_joint_states is None:
            response.success = False
            response.message = "No arm joint state received yet, please try again later"
            return response
        if not self.move_by_odom_client.wait_for_server(timeout_sec=3.0):
            response.success = False
            response.message = "move_by_odom action server not available"
            return response

        self._trigger_count += 1
        # Odd trigger: chassis forward + end-effector backward (-x); even trigger: chassis backward + end-effector forward (+x)
        arm_direction = -1.0 if (self._trigger_count % 2 == 1) else 1.0
        base_direction = 'forward' if arm_direction < 0.0 else 'backward'
        phase = 1 if arm_direction < 0.0 else 2

        self._running = True
        threading.Thread(
            target=self._combined_worker,
            args=(phase, arm_direction, base_direction), daemon=True
        ).start()

        arm_desc = "backward" if arm_direction < 0.0 else "forward"
        response.success = True
        response.message = (
            f"Phase {phase} started: chassis {base_direction} {self.base_distance:.2f}m, "
            f"arm end-effector moves {arm_desc} {self.arm_distance:.2f}m simultaneously"
        )
        self.get_logger().info(response.message)
        return response

    # ========== Coordinated motion worker thread ==========
    def _combined_worker(self, phase, arm_direction, base_direction):
        try:
            self._combined_move_impl(phase, arm_direction, base_direction)
        except Exception as e:
            self.get_logger().error(f"Combined move thread exception: {e}")
        finally:
            self._running = False

    def _combined_move_impl(self, phase, arm_direction, base_direction):
        base_j2, base_j3, base_j4 = float(self.init_pose[1]), float(self.init_pose[2]), float(self.init_pose[3])
        fixed_j1, fixed_j5, fixed_j6 = int(self.init_pose[0]), int(self.init_pose[4]), int(self.init_pose[5])
        # Target extension amount: backward to -arm_distance, forward back to 0 (frontmost), transitioning continuously from the current extension amount without returning to the initial position
        start_disp = self._arm_disp
        end_disp = -self.arm_distance if arm_direction < 0.0 else 0.0
```

```
        self.get_logger().info(
            f"Phase {phase}: base joints J2={base_j2:.1f}, J3={base_j3:.1f}, J4=
{base_j4:.1f}, "
            f"arm displacement {start_disp:+.3f}m -> {end_disp:+.3f}m"
        )

        # PID and smoother state is kept across triggers, so the robotic arm moves
continuously from the current pose
        self._moved_distance = 0.0
        goal = MoveByOdom.Goal()
        goal.distance = float(abs(self.base_distance))
        goal.angle = 0.0
        goal.direction = base_direction
        try:
            send_goal_future = self.move_by_odom_client.send_goal_async(
                goal, feedback_callback=self._odom_feedback_callback
            )
            while rclpy.ok() and not send_goal_future.done():
                time.sleep(0.05)
            goal_handle = send_goal_future.result()
            if not goal_handle.accepted:
                self.get_logger().warn("MoveByOdom goal rejected by action server")
                return
            result_future = goal_handle.get_result_async()
        except Exception as e:
            self.get_logger().error(f"Send MoveByOdom goal exception: {e}")
            return
        self.get_logger().info(f"Chassis start moving {base_direction}
{self.base_distance:.2f}m, arm moving simultaneously")

        # While the chassis is moving, the extension target transitions linearly from the
current value to the target value (interpolated by chassis progress)
        while rclpy.ok() and not result_future.done():
            progress = self._moved_distance / self.base_distance
            progress = max(0.0, min(progress, 1.0))
            self._update_arm_reach(start_disp + (end_disp - start_disp) * progress,
                                   base_j2, base_j3, base_j4, fixed_j1, fixed_j5, fixed_j6)
            time.sleep(self.arm_control_period)

        # Chassis in position, confirm the action result and ensure the end-effector reaches
the final target
        try:
            result = result_future.result().result
            if result.success:
                self.get_logger().info(f"Chassis movement complete: {result.message}")
            else:
                self.get_logger().warn(f"Chassis movement failed: {result.message}")
        except Exception as e:
            self.get_logger().error(f"MoveByOdom result exception: {e}")

        # After the chassis is in position, keep issuing until the smoother converges to the
final extension amount (the smoother has a limited step, so it needs to catch up)
        final_displacement = end_disp
```

```
        converge_deadline = time.time() + 5.0
        while rclpy.ok() and time.time() < converge_deadline:
            raw_z = self.reach_pid.compute(final_displacement, 0.0)
            self._update_arm_reach(final_displacement,
                                   base_j2, base_j3, base_j4, fixed_j1, fixed_j5, fixed_j6,
runtime=500)
            if abs(raw_z - self.last_angular_z) < 0.005:
                break
            time.sleep(self.arm_control_period)
        self._arm_disp = end_disp
        time.sleep(1.5)
        self.get_logger().info(f"Phase {phase} combined motion complete")

    def _odom_feedback_callback(self, feedback_msg):
        self._moved_distance = feedback_msg.feedback.moved_distance

    def _update_arm_reach(self, displacement, base_j2, base_j3, base_j4,
                          fixed_j1, fixed_j5, fixed_j6, runtime=None):
        """Update J2/J3/J4 according to the target extension amount: same mapping as
apriltag_follow.py, only the input differs"""
        if runtime is None:
            runtime = self.arm_step_runtime
        raw_z = self.reach_pid.compute(displacement, 0.0)

        # Smoothing (Slew Rate Limiter)
        diff_z = raw_z - self.last_angular_z
        if diff_z > self.max_step_change_z:
            angular_z = self.last_angular_z + self.max_step_change_z
        elif diff_z < -self.max_step_change_z:
            angular_z = self.last_angular_z - self.max_step_change_z
        else:
            angular_z = raw_z
        self.last_angular_z = angular_z

        target_j2 = base_j2 + angular_z * self.gain_reach_j2
        target_j3 = base_j3 + angular_z * self.gain_reach_j3
        target_j4 = base_j4 + angular_z * self.gain_reach_j4

        joints = [
            fixed_j1,
            int(max(0, min(180, target_j2))),
            int(max(0, min(180, target_j3))),
            int(max(0, min(180, target_j4))),
            fixed_j5,
            fixed_j6
        ]
        self.pubSix_Arm(joints, runtime=runtime)
        self._current_joints_cmd = joints
```